

Architettura Cloud-Native per un Sistema di Retrieval Basato su Grafi Semantici

Sistemi Cloud

Santi Lisi

Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Sommario

Il presente elaborato descrive la progettazione e l'implementazione dell'infrastruttura cloud a supporto di un sistema avanzato di retrieval semantico basato su grafi. Il progetto originale esegue l'inferenza sfruttando un Vision-Language Model (VLM), Nomic e una Graph Neural Network (GCN). Data l'elevata complessità computazionale, è emersa la necessità di reingegnerizzare il sistema monolitico in una moderna architettura a microservizi asincroni. La relazione documenta il percorso evolutivo dell'infrastruttura, partendo da un ambiente containerizzato locale, passando per un cluster Kubernetes locale (K3s), fino a giungere a una soluzione Cloud-Native su Amazon Web Services (AWS) automatizzata tramite Terraform. L'architettura finale su AWS integra servizi completamente gestiti (S3, SQS, RDS, ElastiCache) e garantisce una scalabilità elastica e reattiva per le immagini pre-calcolate. Per le nuove immagini immesse dall'utente, il sistema sfrutta le code SQS per delegare asincronamente i task computazionalmente intensivi (pipeline VLM) al cluster HPC universitario, gestendo l'accesso concorrente alle risorse limitate tramite semafori distribuiti.

Parole chiave: cloud computing, microservizi, aws, terraform, k3s, ansible, docker, architettura distribuita

Indice

1	Introduzione	4
1.1	L'Obiettivo del Progetto: Dal Monolite al Cloud-Native .	5
1.2	Analisi dei Requisiti Architettureali	5
1.3	Struttura della Trattazione	6
2	Applicativo, Pipeline AI e Ibridazione HPC	7
2.1	La Pipeline Asincrona	8
2.2	Ibridazione Cloud-HPC: Il Problema del Collo di Bottiglia	9
2.2.1	Sincronizzazione tramite Distributed Lock (Redis Semaforo)	10
3	Containerizzazione e Orchestrazione Locale	11
3.1	La Containerizzazione con Docker e il Decoupling	12
3.2	Simulazione di una Topologia Multi-Nodo tramite Multipass	13
3.3	Scelta di K3s come Distribuzione Kubernetes	14
3.4	Automazione del Cluster con K3s e Ansible	15
3.5	Gestione del Ciclo di Vita e Dettagli Implementativi . . .	16
3.5.1	Comandi Operativi per l'Avvio	17
3.5.2	Ciclo di Vita e Persistenza Locale	18
3.5.3	Scaling Manuale dei Pod in Locale	19
3.5.4	Evoluzione del Playbook: Dall'Approccio Ibrido alla Separazione	19
4	Infrastruttura AWS e Servizi Gestiti	20
4.1	Infrastructure as Code con Terraform	20
4.2	Topologia di Rete e Sicurezza (VPC)	21
4.3	Offloading verso Servizi Gestiti AWS (PaaS/SaaS)	22
4.4	Risoluzione Criticità Hardware: Out-Of-Memory (OOM)	23
4.5	Innesco Serverless (Event-Driven) tramite AWS Lambda	24
5	Deployment K3s Cloud ed Auto-Scaling Elastico	24
5.1	Gestione dei Secret e Manifesti Kubernetes	25

5.2	Event-Driven Auto-Scaling con KEDA	25
5.3	Continuous Integration e Continuous Deployment (CI/CD)	27
5.4	Riproducibilità e Automazione del Provisioning	28
6	Conclusioni e Sviluppi Futuri	29
6.1	Sviluppi Futuri	30

1 Introduzione

Il presente progetto si pone l'obiettivo di ingegnerizzare e portare in produzione un sistema avanzato di ricerca semantica tra grafi che descrivono immagini. A differenza dei tradizionali motori di ricerca visiva basati sull'apparenza dei pixel e sulle texture, questo applicativo si fonda sulla *similarità compositiva*: la ricerca avviene confrontando la topologia semantica delle scene, ovvero gli oggetti presenti e le relazioni logico-spaziali che li connettono, formalizzati sotto forma di Grafi di Scena (Scene Graphs).

L'applicativo web sviluppato permette agli utenti di esplorare le capacità del modello neurale in due modalità principali: testando il sistema su immagini provenienti da un test set predefinito, oppure caricando nuove immagini personali. In risposta a una query, i risultati vengono mostrati graficamente e confrontati in tempo reale con altri sistemi di ricerca (modelli *baseline* visivi). L'utente ha la possibilità di analizzare le k-immagini più simili recuperate e di ispezionare visivamente il confronto tra il grafo generato dalla query e i grafi dei risultati estratti dalla galleria.

Per l'elaborazione delle nuove immagini caricate, il sistema integra una complessa pipeline associata a un Vision-Language Model (VLM). Questa pipeline si occupa di processare l'immagine sorgente, estrarre il grafo semantico corrispondente e proiettarlo nello spazio metrico di ricerca per ottenere i risultati finali.

Tuttavia, l'enorme potenza espressiva di queste architetture neurali porta con sé un costo computazionale formidabile. L'elaborazione di una query visiva richiede il caricamento in memoria GPU di modelli ad alta capacità e cicli di inferenza lenti. In un contesto web di produzione, esporre tali funzionalità tramite un'architettura tradizionale monolitica risulta inefficiente: i lunghi tempi di inferenza causerebbero *timeout* delle connessioni HTTP e il blocco delle risorse del web server.

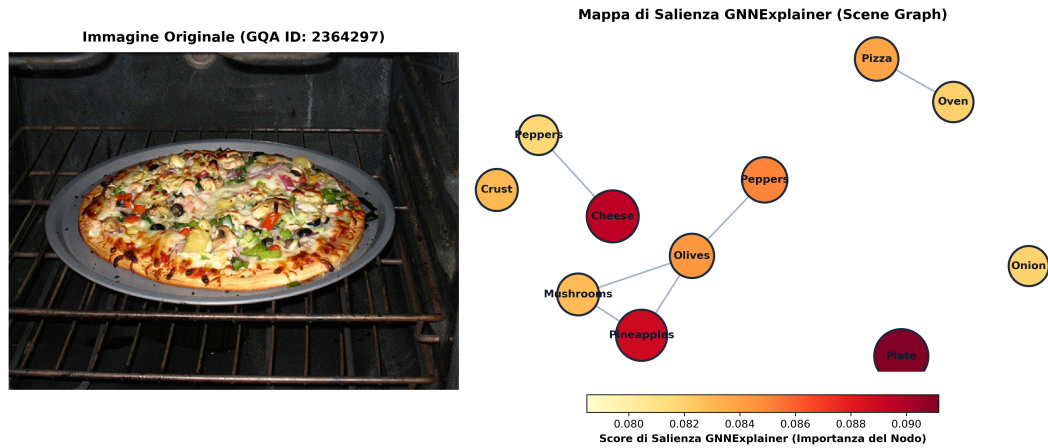


Figura 1: Esempio di estrazione e confronto di grafi semantici: a sinistra l’immagine originale e i risultati, a destra i grafi di scena che catturano le relazioni logico-spaziali degli oggetti.

1.1 L’Obiettivo del Progetto: Dal Monolite al Cloud-Native

Al fine di superare questo collo di bottiglia, l’obiettivo fondante di questo progetto   stato quello di reingegnerizzare in ottica *Cloud-Native* un’applicazione di elaborazione grafica e semantica inizialmente concepita per un’esecuzione locale e monolitica. Il sistema   stato suddiviso in microservizi indipendenti e riprogettato secondo i principi delle architetture asincrone (Event-Driven Architecture), rendendolo idoneo per il deployment su infrastrutture Cloud di livello enterprise, come Amazon Web Services (AWS).

Questa migrazione architetturale ha comportato una riscrittura dei flussi logici, introducendo la separazione formale tra il gateway web (incaricato esclusivamente di ricevere le richieste e servire l’interfaccia utente) e il motore di calcolo, denominato AI Worker.

1.2 Analisi dei Requisiti Architettureali

La progettazione dell’architettura distribuita ha mirato a soddisfare specifici requisiti non funzionali e pattern ingegneristici del Cloud Computing:

- *Asincronia e Decoupling*: Si è reso indispensabile il disaccoppiamento (decoupling) logico tra il ciclo di vita della richiesta dell'utente e l'elaborazione AI sottostante. Il decoupling garantisce che un malfunzionamento o un sovraccarico nel sistema di inferenza non comprometta la disponibilità del server web frontale. Ciò è stato realizzato attraverso l'adozione di Message Broker (code di messaggistica) che agiscono da cuscinetto asincrono tra i microservizi.
- *Scalabilità Orizzontale Elastica*: Il traffico in ingresso può presentare variazioni imprevedibili. Lo strato di Worker che esegue le pipeline neurali deve avere la capacità di effettuare scale-out e contrarsi dinamicamente in proporzione diretta alla lunghezza della coda di messaggi in sospeso.
- *Isolamento e Containerizzazione Stateless*: Al fine di prevenire conflitti tra dipendenze di sistema e garantire riproducibilità, l'intero applicativo è stato compartimentato in container Docker. I container sono stati progettati per essere rigorosamente stateless, ovvero privi di stato interno salvato su disco. Lo stato del sistema e i dati persistenti sono delegati interamente a database gestiti esterni, permettendo la distruzione o la clonazione dei container in qualsiasi momento senza perdite di dati.
- *Infrastructure as Code (IaC)*: Ogni singola risorsa infrastrutturale non è stata configurata manualmente tramite interfaccia grafica. Il deployment è stato codificato in script dichiarativi, rendendolo un processo deterministico.

1.3 Struttura della Trattazione

La relazione dell'architettura cloud sviluppata seguirà una struttura divisa per livelli di astrazione:

1. *Applicativo, Pipeline AI e Ibridazione HPC* (Capitolo 2): Il secondo

capitolo esplorerà il funzionamento interno della pipeline neurale. Particolare enfasi verrà posta sulla natura ibrida della soluzione: a causa dei limiti nel caricare on-demand modelli di grandi dimensioni sul cloud, il sistema delega i task computazionalmente più onerosi (VLM) verso il cluster HPC dell'università. Sarà descritta la soluzione del Distributed Lock usata per serializzare l'accesso alle risorse limitate di tale cluster.

2. *Orchestrazione Locale* (Capitolo 3): Verrà documentata la fase di orchestrazione *on-premise*. Sarà mostrato come i servizi siano stati virtualizzati sul computer locale sfruttando Multipass e gestiti tramite una distribuzione leggera di Kubernetes (K3s), automatizzata tramite Ansible.
3. *Infrastruttura AWS* (Capitolo 4): Il quarto capitolo analizzerà l'infrastruttura di Amazon Web Services (AWS). Verrà illustrato il codice Terraform impiegato per configurare una Virtual Private Cloud (VPC) sicura e per delegare i datastore e i message broker all'offerta PaaS nativa di AWS (RDS, SQS, ElastiCache e S3).
4. *Deployment ed Auto-Scaling Elastico* (Capitolo 5): L'ultimo capitolo analizzerà i manifesti di Kubernetes utilizzati per lanciare i microservizi all'interno del cluster K3s su AWS. Si focalizzerà sull'orchestrazione asincrona, sull'innescio guidato da AWS Lambda e sull'implementazione dell'Auto-Scaling elastico pilotato dagli eventi tramite KEDA.

2 Applicativo, Pipeline AI e Ibridazione HPC

Il cuore logico e funzionale di questo progetto è rappresentato da un sofisticato motore di ricerca semantico visivo. L'intento dell'applicativo è consentire a un utente finale di caricare un'immagine dal proprio dispositivo e, attraverso una catena di elaborazione neurale, individuare le immagini concettualmente e strutturalmente più simili all'interno di un

vasto database pre-calcolato.

A differenza delle tradizionali pipeline di *Image Retrieval* basate esclusivamente sull'estrazione di *feature* visive globali (es. architetture ResNet o ViT standard), questo sistema adotta un approccio avanzato incentrato sui *Grafi di Scena* (*Scene Graphs*). Un grafo di scena modella l'immagine come un insieme di nodi (che rappresentano entità o oggetti) connessi da archi (che codificano le relazioni spaziali, semantiche o logiche tra di essi). Per generare tali grafi a partire da immagini non strutturate, il sistema si affida a potenti *Vision-Language Models* (VLM). Nello specifico, il progetto fa uso del modello Qwen2.5-VL-7B-Instruct, una versione all'avanguardia dotata di 7 miliardi di parametri, in grado di comprendere complessi contesti visivi e istruzioni testuali. Una volta ottenuto il grafo, quest'ultimo viene proiettato in uno spazio vettoriale continuo (embedding latente) tramite l'uso di una *Graph Convolutional Network* (GCN), nello specifico un'architettura GINE con Triple Margin Loss o varianti customizzate. L'embedding risultante viene infine confrontato con quelli memorizzati in un Vector Database ad alte prestazioni.

2.1 La Pipeline Asincrona

Per sostenere l'oneroso ciclo di calcolo descritto senza incorrere in timeout lato client, l'intera pipeline di elaborazione è stata progettata seguendo il paradigma architetturale dell'asincronia guidata dagli eventi (*Event-Driven*). Il flusso logico si articola nelle seguenti fasi:

1. *Upload e Intercettazione*: L'utente, tramite l'interfaccia React (Frontend), seleziona un'immagine. L'immagine non viene inviata al Backend tramite una tipica chiamata HTTP in base64 (che saturerebbe la banda e la memoria), ma viene caricata direttamente sullo storage a oggetti (S3 o una sua simulazione locale).
2. *Creazione del Job e Accodamento*: Il Backend (FastAPI), che espone il livello *RESTful API*, riceve la richiesta tramite l'endpoint `POST /api/v1/search`. A questo punto registra un nuovo "Job"

assegnandogli un ID univoco in formato UUID e salvandone lo stato iniziale ("PROCESSING") sul database relazionale (PostgreSQL). Simultaneamente, il payload della richiesta (contenente il path dell'immagine e i metadati della ricerca) viene inserito in coda di messaggistica (RabbitMQ in locale, AWS SQS sul Cloud).

3. *Polling Asincrono*: A questo punto la chiamata HTTP del client si chiude con successo, restituendo immediatamente l'ID del Job. Il frontend attiva una routine di *polling* leggero, interrogando il backend tramite l'endpoint `GET /api/v1/status/{job_id}` a intervalli regolari (es. ogni 1 secondo) esclusivamente per leggere lo stato del Job (es. "SCENE_GRAPH", "VECTOR_SEARCH", "COMPLETED") recuperandolo da un livello di cache ultraveloce (Redis), evitando di sovraccaricare il DB relazionale.
4. *Elaborazione del Worker*: Un componente separato, l'AI Worker, resta perennemente in ascolto sulla coda di messaggistica. Non appena percepisce la presenza di un messaggio, lo estrae (de-queue) e avvia la complessa pipeline neurale. Tutte le transizioni di stato della pipeline vengono istantaneamente riflesse su Redis, aggiornando in tempo reale la UI dell'utente.

2.2 Ibridazione Cloud-HPC: Il Problema del Collo di Bottiglia

Durante la migrazione verso l'infrastruttura Cloud-Native, è emerso un limite fisico ed economico intrinseco alle infrastrutture di Cloud Pubblico: il costo orario per il provisioning di nodi Kubernetes equipaggiati con GPU di fascia altissima (necessarie per far girare modelli VLM da svariati miliardi di parametri) è economicamente proibitivo in un contesto accademico.

Per ovviare a questo problema, si è optato per una soluzione di *Ibridazione Cloud-HPC*. L'infrastruttura Cloud (economica, elastica e basata

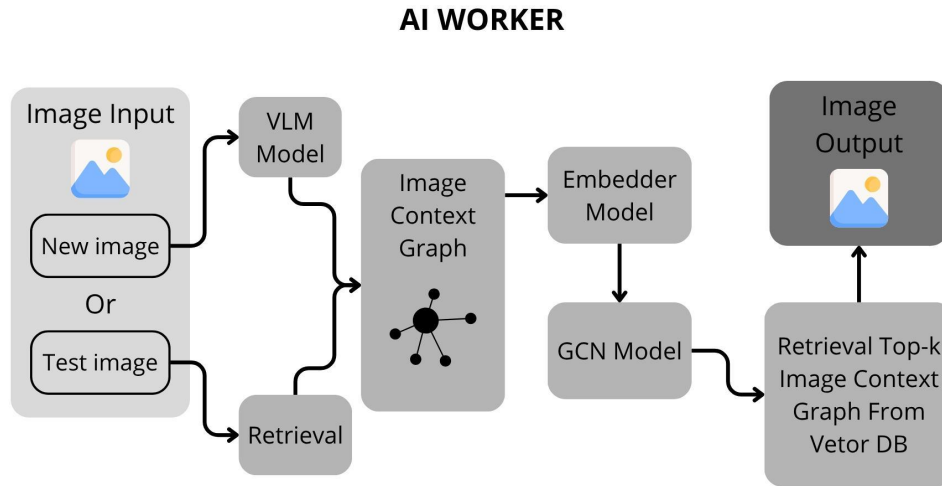


Figura 2: Architettura asincrona e interazione tra i componenti della pipeline AI (Frontend, Backend, Redis, RabbitMQ/SQS e AI Worker).

su CPU) ospita l'intero strato web e di orchestrazione, mentre il calcolo intensivo (l'estrazione del Grafo di Scena tramite VLM) viene delegato asincronamente al cluster *High Performance Computing* (HPC) fornito dall'Università. Il Worker Cloud, tramite una connessione SSH automatizzata (utilizzando `paramiko`), trasferisce l'immagine, compila e sottomette uno script SLURM (`sbatch`) al cluster HPC, e attende passivamente il download dell'output testuale contenente il grafo.

2.2.1 Sincronizzazione tramite Distributed Lock (Redis Semaforo)

Tale architettura ibrida introduce tuttavia una complessa sfida nei sistemi distribuiti. Mentre l'Auto-Scaling del Cloud è in grado di spawnare decine di pod Worker in parallelo all'aumentare delle richieste (come verrà approfondito nel Capitolo 5), il cluster HPC a valle rappresenta un *collo di bottiglia rigido*: le risorse (GPU) assegnate al progetto sul cluster universitario permettono l'esecuzione di una sola istanza del modello VLM per volta. Se due pod Worker Cloud tentassero simultaneamente di avviare il job SLURM sull'HPC, il secondo andrebbe in errore per esaurimento risorse (Out-of-Memory).

Per risolvere in modo efficiente questo collasso infrastrutturale garantendo contemporaneamente il massimo parallelismo, è stato implementato un *Distributed Lock* (Semaforo Distribuito) basato su Redis. La logica implementata nel Worker è la seguente:

- Prima di aprire la connessione SSH verso l'HPC, il Worker tenta di acquisire una chiave univoca in Redis (`SETNX lock:hpc_gpu`).
- Se l'operazione ha successo, il Worker acquisisce il lock esclusivo e procede all'inferenza VLM sull'HPC.
- Se un secondo pod Worker (generato dall'autoscaling) tenta di elaborare un'altra immagine parallelamente, fallirà l'acquisizione del lock. Aniché terminare con un errore, il pod entrerà in un ciclo di *wait* asincrono, riprovando l'acquisizione ogni 5 secondi, mettendo temporaneamente in pausa l'elaborazione del suo specifico task.
- Al termine del job HPC, il primo Worker rilascia la chiave (`DEL`), sbloccando istantaneamente il Worker in attesa.

Questo pattern architetturale, tipico delle infrastrutture *enterprise*, ha permesso di scindere le operazioni scalabili da quelle contingentate. Le fasi di download delle immagini, il preprocessing, l'inferenza di modelli Baseline (es. ResNet/CLIP) e la Vector Search vengono eseguite su decine di pod in parallelo puro; l'accesso alla GPU dell'HPC viene invece serializzato dinamicamente, garantendo tolleranza ai guasti e resilienza totale senza alcuno spreco di risorse Cloud.

3 Containerizzazione e Orchestrazione Locale

Una transizione efficace verso il paradigma Cloud-Native impone, come requisito fondamentale, l'indipendenza assoluta dell'applicativo dall'ambiente hardware e software sottostante. Al fine di scongiurare la nota problematica della dipendenza dall'ambiente di sviluppo e per garantire

un deployment privo di attriti, il primo passo implementativo è consistito in una rigorosa containerizzazione dell'intera suite applicativa.

Successivamente, prima di allocare risorse su un Cloud pubblico, l'intera architettura distribuita è stata modellata e collaudata *on-premise* (in locale). A tal fine si è proceduto alla simulazione di un vero e proprio data-center virtuale all'interno di una singola workstation, utilizzando tecnologie di virtualizzazione leggere (Multipass) e di orchestrazione (K3s e Ansible).

3.1 La Containerizzazione con Docker e il Decoupling

L'applicazione monolitica originale è stata suddivisa applicando il principio del *decoupling* (disaccoppiamento). Il decoupling consiste nel separare le responsabilità logiche di un'applicazione complessa in componenti più piccoli e indipendenti (microservizi). Questo isolamento garantisce che i componenti possano scalare, essere aggiornati o persino fallire in modo indipendente l'uno dall'altro, senza causare il crollo dell'intero sistema. Nello specifico, il sistema è stato strutturato in tre microservizi primari, ciascuno isolato all'interno di una specifica immagine Docker:

- *Frontend Container*: Un'applicazione React (Vite) pacchettizzata insieme a un server NGINX ad alte prestazioni per il serving dei file statici. L'immagine Docker è stata costruita mediante un approccio *multi-stage build* per ridurre il peso finale (scartando le dipendenze non necessarie in fase di runtime).
- *Backend Container*: Un server Python basato sul framework asincrono FastAPI. Questo container espone unicamente l'endpoint di ricezione, registra lo stato iniziale della richiesta sul database e innesca la produzione di messaggi verso il broker.
- *AI Worker Container*: L'elemento computazionalmente intensivo, contenente lo stack PyTorch, le librerie CUDA, FAISS (per la simi-

larità vettoriale) e le dipendenze di rete. Il Worker resta in ascolto in modalità *long-polling* sulla coda messaggi, isolando così i calcoli gravosi dalle chiamate web dell'utente.

I container implementati sono stati progettati per essere rigorosamente *stateless*. Un container stateless è un ambiente di esecuzione volatile che non salva alcun dato o stato persistente nel proprio file system locale. Di conseguenza, il ciclo di vita del container non è vincolato ai dati dell'utente: il container può essere distrutto, riavviato o clonato dall'autoscaler in qualsiasi istante senza alcuna perdita di informazioni. L'intero stato del sistema, le identità dei task e i risultati vengono invece delegati a database esterni completamente separati: *PostgreSQL* funge da database relazionale per mantenere la traccia permanente delle operazioni, mentre *Redis* funge da archivio *in-memory* ad alte prestazioni per la sincronizzazione rapida e la memorizzazione dei lock distribuiti. L'utilizzo di code di messaggistica (*RabbitMQ*) garantisce inoltre la resilienza delle richieste in transito tra i microservizi.

3.2 Simulazione di una Topologia Multi-Nodo tramite Multipass

Al fine di validare l'architettura Kubernetes prima del deploy in cloud su AWS, si è deciso di ricreare un ambiente di orchestrazione multi-nodo in locale. Essendo impraticabile l'uso di cluster fisici, si è optato per l'impiego di Multipass, uno strumento sviluppato da Canonical che consente l'orchestrazione rapida di macchine virtuali Ubuntu su hypervisor nativi.

Attraverso script di *provisioning* custom (`setup_multipass.sh`), sono state generate tre macchine virtuali logicamente separate:

1. **k3s-master**: Il nodo di Control Plane, incaricato di esporre le API Server di Kubernetes e prendere le decisioni di scheduling.
2. **k3s-worker-1** e **k3s-worker-2**: I nodi di elaborazione fisica, designati all'esecuzione effettiva dei container (Pod).

Questa compartimentazione ha permesso di collaudare le dinamiche di rete interna del cluster, le policy di rete e lo scheduling multi-nodo, mettendo alla prova la robustezza del sistema di fronte alla caduta intenzionale di uno dei nodi Worker.

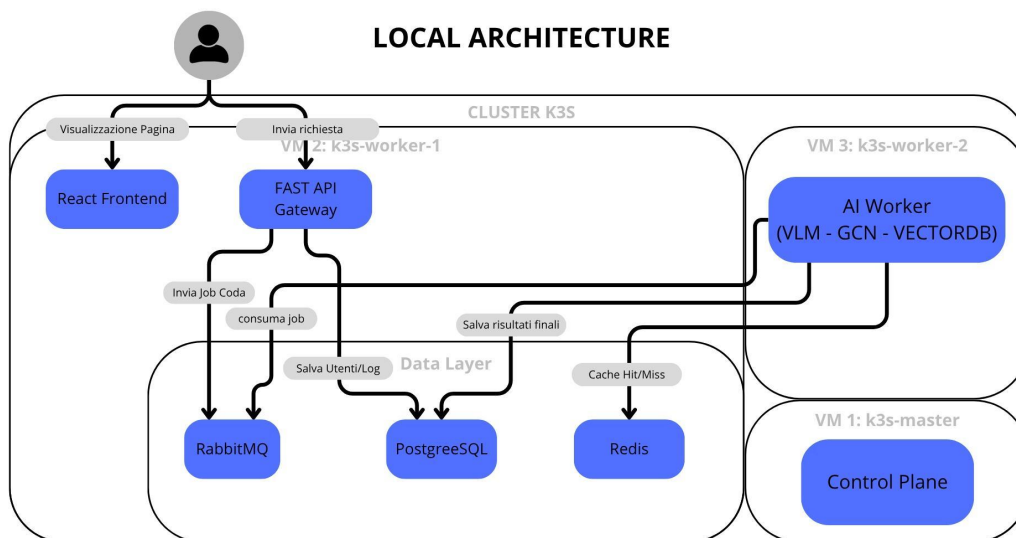


Figura 3: Architettura locale di test basata su macchine virtuali (Multipass) orchestrate con K3s.

3.3 Scelta di K3s come Distribuzione Kubernetes

La scelta di adottare K3s per l'ambiente locale (Multipass), al posto della distribuzione vanilla di Kubernetes (K8s) utilizzata invece nel cloud AWS, è stata motivata da stringenti limiti di risorse hardware del computer di sviluppo.

Kubernetes standard (K8s), che è stato impiegato con successo (spinto al limite) sulle istanze EC2 di AWS tramite `kubeadm`, è progettato per ambienti enterprise: il suo *control plane* richiede almeno 2 CPU e 2 GB di RAM per il solo nodo master, utilizza `etcd` come backend distribuito e include numerosi controller. Replicare per intero la stessa gravosità di K8s su macchine virtuali locali avrebbe saturato istantaneamente la workstation.

K3s risolve questo problema in fase di sviluppo essendo una distribuzione Kubernetes certificata CNCF, ma alleggerita:

- Sostituisce *etcd* con SQLite come backend per il *Control Plane*, riducendo il consumo di risorse del nodo master di circa il 70%.
- Rimuove driver e funzionalità deprecate (alpha features, driver in-tree per cloud provider storici) non necessarie per questo scenario.
- Compila l'intero sistema in un singolo eseguibile da circa 60 MB, rendendo l'installazione e l'aggiornamento estremamente rapidi.
- Supporta la stessa API di Kubernetes standard, garantendo la piena compatibilità.

Il principio guida è stato la *parità semantica*: si usa K3s in locale perché è *resource-efficient*, ma con la garanzia matematica che i manifesti YAML (Deployment, Service, Ingress, ecc.) siano identici e trasferibili senza alcuna modifica al cluster K8s puro (basato su *kubeadm*) in produzione su AWS.

3.4 Automazione del Cluster con K3s e Ansible

Per trasformare le macchine virtuali in un cluster Kubernetes pienamente operativo, l'installazione manuale è stata scartata a favore di una soluzione automatizzata e ripetibile. Si è scelto K3s, una distribuzione di Kubernetes certificata e alleggerita, ottimizzata eliminando driver legacy e sostituendo *etcd* con un più leggero database SQLite.

Per evitare qualsiasi interazione manuale, il processo di bootstrap è stato codificato in un *playbook* Ansible. L'architettura IaC (Infrastructure as Code) ha permesso di:

- Aggiornare e mettere in sicurezza i pacchetti di sistema operativo su tutte le macchine virtuali in parallelo.
- Installare e configurare le chiavi SSH e le regole di firewall (*ufw*) necessarie alla comunicazione intra-cluster.

- Eseguire in modo asincrono lo script di installazione del nodo Master, estrapolarne il token crittografico generato, per poi iniettarlo in maniera dinamica nei nodi Worker per avviare il processo di unione al cluster.

Al termine dell'esecuzione del *playbook* Ansible, l'ambiente di sviluppo restituisce in automatico il file `kubeconfig` completo, fornendo agli operatori un cluster Kubernetes multi-nodo funzionante, isolato e facilmente ricreabile con un solo comando, gettando le fondamenta per il successivo collaudo del codice di *cloud deployment*.

3.5 Gestione del Ciclo di Vita e Dettagli Implementativi

La natura dinamica di un ambiente di sviluppo locale richiede soluzioni custom per garantire la fluidità del ciclo di vita del cluster. In questo contesto, abbiamo sviluppato e implementato script Bash dedicati per simulare il comportamento di un vero approccio *Infrastructure as Code* e risolvere problematiche tecniche specifiche:

- Gestione degli IP Dinamici (`update_cluster_ip.sh`): Essendo i nodi VM gestiti tramite Multipass, l'hypervisor assegna indirizzi IP dinamici ad ogni riavvio della macchina host. Questo romperebbe la connettività del file `kubeconfig` (che punta all'IP del Master) e la capacità del Frontend React di comunicare con le API (configurate tramite `VITE_API_URL`). Lo script `update_cluster_ip.sh` intercetta dinamicamente il nuovo IP, aggiorna le configurazioni locali e inietta il nuovo valore nei manifest Kubernetes, automatizzando il riallineamento del cluster senza alcun intervento manuale.
- Strategia di Containerizzazione e `containerd`: K3s, in ottemperanza ai moderni standard CRI (*Container Runtime Interface*) di Kubernetes, non utilizza più il pesante demone Docker, bensì `containerd`. Pertanto, per popolare i nodi con le nostre immagini ap-

plicative senza sprecare risorse per un *Container Registry* locale e senza gravare sulla banda per il pull da Docker Hub, abbiamo implementato un sistema di *sideloading* (eseguito dallo script `load_images_k3s.sh`). Le immagini vengono prima compilate e salvate in un archivio tarball compresso; successivamente, lo script utilizza `ctr` (l'interfaccia client nativa di containerd) per importare istantaneamente l'archivio direttamente nel namespace K8s di ogni singolo nodo.

- Scelta delle Porte e NodePort: I servizi sono esposti all'esterno tramite *NodePort*. È interessante notare che Kubernetes impone un range restrittivo (30000-32767) per l'apertura delle porte, per evitare conflitti con porte privilegiate dell'host. Abbiamo mappato mnemonicamente le porte traslando quelle originali: il Frontend (porta originaria 5173) viene esposto sulla 30173, mentre il Backend FastAPI (porta originaria 80/8000) viene esposto sulla 30080.

3.5.1 Comandi Operativi per l'Avvio

Per l'orchestrazione locale è sufficiente lanciare in sequenza gli script predisposti. Il primo comando legge automaticamente la configurazione dal file `project_config.env` situato nella cartella radice del progetto, avvia la creazione delle VM tramite Multipass e prepara l'inventario per Ansible:

```
$ cd cloud/deploy/local
$ ./setup_multipass.sh
```

Successivamente, si esegue il playbook Ansible per installare K3s sui nodi appena creati:

```
$ cd ../../ansible
$ ansible-playbook -i inventory/hosts.ini site.yml
```

Una volta conclusa l'installazione, si torna nella cartella di deploy, si allineano gli IP dinamici delle macchine (aggiornando anche i manifest) e si esporta la configurazione generata per interagire con il cluster:

```
$ cd ../deploy/local
$ ./update_cluster_ip.sh
$ export KUBECONFIG=$(pwd)/kubeconfig
$ kubectl get nodes
```

Il cluster a questo punto è operativo ma vuoto. Il passo successivo consiste nel caricare le immagini Docker delle applicazioni all'interno del registro locale di ogni nodo. Poiché le VM non hanno accesso diretto al Docker Hub di produzione, le immagini vengono prima esportate in un archivio `.tar` e poi importate manualmente in ciascun nodo tramite uno script dedicato:

```
$ ./load_images_k3s.sh
```

Infine, si applicano al cluster i manifesti Kubernetes (Deployment, Service, ConfigMap, etc.) contenuti nella cartella `k8s/`, avviando così tutti i microservizi del sistema (frontend, backend, worker, database, broker):

```
$ kubectl apply -f k8s/
```

A questo punto, trattandosi di un database appena creato e inizialmente vuoto, è necessario attendere che il pod del backend sia in stato **Running** e successivamente lanciare il comando di inizializzazione per popolare le tabelle:

```
$ kubectl exec deployment/backend -- python -c "from main import init_db; init_db()"
```

Per verificare che tutti i Pod siano partiti correttamente:

```
$ kubectl get pods
```

3.5.2 Ciclo di Vita e Persistenza Locale

Per mettere in pausa l'infrastruttura locale senza dover ricreare l'intero cluster, è possibile arrestare e riavviare le macchine virtuali:

```
$ multipass stop --all  
$ multipass start --all
```

Kubernetes è un sistema intrinsecamente persistente: il database etcd/SQLite preserva lo *Stato Desiderato*. Al riavvio delle macchine, non è necessario riapplicare i manifesti YAML (`kubectl apply`) né ricaricare le immagini Docker o re-inizializzare il database Postgres. I nodi worker ripristineranno autonomamente i pod attivi prima dello spegnimento. L'unica operazione necessaria ad ogni riavvio è l'esecuzione dello script `update_cluster_ip.sh` (discusso in precedenza) per allineare le configurazioni al nuovo IP dinamico assegnato dal server DHCP del sistema host.

Al termine definitivo del progetto, l'intero cluster locale può essere rimosso in modo permanente eliminando le VM allocate sulla workstation:

```
$ multipass delete k3s-master k3s-worker-1 k3s-worker-2 --purge
```

3.5.3 Scaling Manuale dei Pod in Locale

A differenza del Cloud (dove KEDA gestisce lo scaling automatico in base alla profondità della coda SQS), in locale lo scaling avviene a livello di *Pod*, non di nodi VM. Kubernetes distribuisce automaticamente le repliche sui nodi disponibili. Per aumentare il numero di istanze del worker in esecuzione è sufficiente un singolo comando:

```
# Scala le repliche del worker (es. da 1 a 3 istanze parallele)
$ kubectl scale deployment/worker --replicas=3

# Verifica la distribuzione dei pod sui nodi
$ kubectl get pods -o wide
```

Questo approccio permette di simulare scenari di carico elevato e verificare che lo scheduler Kubernetes distribuisca correttamente le istanze tra i nodi worker disponibili, replicando il comportamento che in produzione viene gestito automaticamente da KEDA.

3.5.4 Evoluzione del Playbook: Dall'Approccio Ibrido alla Separazione

Nella prima stesura del progetto, l'obiettivo era avere un unico playbook universale (`cloud/ansible/site.yml`) in grado di configurare sia l'ambiente AWS che quello Multipass. Per garantire il riutilizzo del codice (*DRY - Don't Repeat Yourself*), è stato implementato un *fallback condizionale* nelle variabili Jinja2:

```
replace: "{{ ansible_host | default(inventory_hostname) }}"
```

Questa geniale riga di codice permetteva ad Ansible di adattarsi dinamicamente: usare l'IP pubblico `ansible_host` se eseguito su AWS (tramite Terraform), o ripiegare sull'`inventory_hostname` (l'IP diretto) se eseguito nel server DHCP locale di Multipass.

Tuttavia, con l'avanzare dello sviluppo, l'infrastruttura Cloud ha iniziato a richiedere configurazioni molto più complesse e specifiche (come l'installazione di Helm, dell'Ingress Controller NGINX e di KEDA) che non avevano senso di esistere in locale. Per mantenere la massima pu-

lizia architetturale ed evitare un file YAML saturo di clausole condizionali (`when: environment == 'aws'`), si è deciso di *biforcare* il codice in due percorsi di deployment isolati: `deploy/local/` e `deploy/aws/`. Il playbook originario ibrido è stato quindi clonato e ripulito dai fallback (`setup_cluster.yml`), specializzandolo esclusivamente per l'ambiente AWS. Questo ha permesso di mantenere i manifesti Kubernetes (`frontend.yml`, `worker.yml`) perfettamente ottimizzati per i rispettivi ambienti, separando nettamente i test di sviluppo dalla topologia di produzione.

4 Infrastruttura AWS e Servizi Gestiti

Il completamento della fase di test *on-premise* ha fornito le garanzie logiche necessarie per procedere al deployment in un ambiente di produzione reale. La piattaforma scelta per ospitare l'applicativo è Amazon Web Services (AWS), il provider cloud leader di mercato. L'obiettivo primario di questa transizione è stato duplice: da un lato, delegare la gestione dei servizi critici (database e code) a soluzioni PaaS/SaaS native per massimizzarne l'affidabilità; dall'altro, istanziare un cluster Kubernetes in grado di scalare dinamicamente le risorse di calcolo.

4.1 Infrastructure as Code con Terraform

La creazione dell'infrastruttura AWS non è stata eseguita manualmente tramite la console web, procedura notoriamente soggetta a errori umani e difficilmente riproducibile. Si è invece adottato un rigoroso approccio *Infrastructure as Code* (IaC) impiegando *HashiCorp Terraform*.

Tramite script dichiarativi (codificati in HCL - HashiCorp Configuration Language), l'intera topologia cloud è stata modellata come codice versionabile. Questo ha garantito la totale idempotenza del deployment: l'infrastruttura può essere distrutta e ricreata in modo identico in pochi minuti, assicurando una coerenza perfetta tra l'ambiente di *staging* e

quello di *production*.

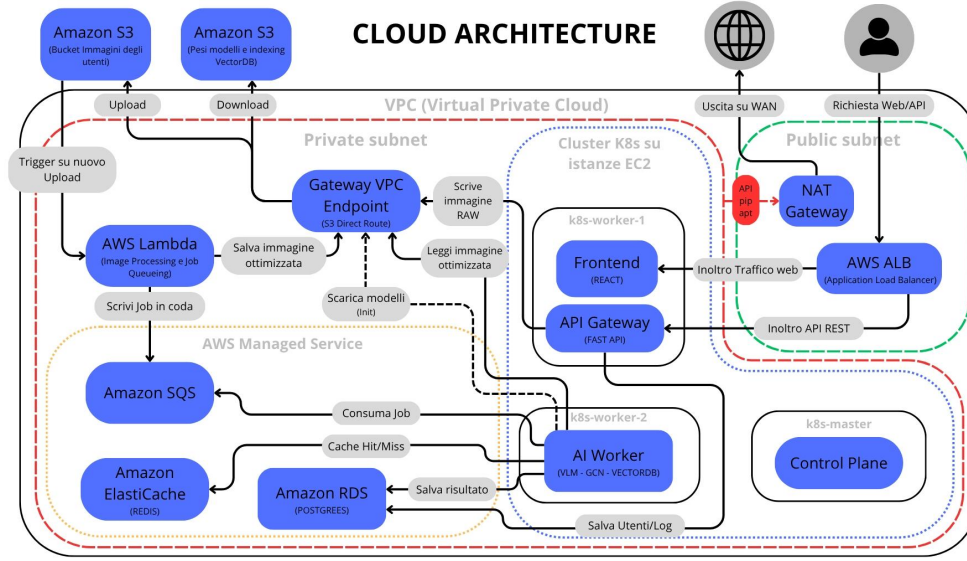


Figura 4: Architettura Cloud su AWS. La VPC ospita i nodi di calcolo K3s (Master e Worker) nella subnet pubblica per consentire il provisioning diretto via Ansible, mentre i dati critici sono isolati nelle subnet private grazie alle offerte fully-managed (RDS, ElastiCache).

4.2 Topologia di Rete e Sicurezza (VPC)

Il livello base dell'architettura AWS è costituito da una *Virtual Private Cloud (VPC)*, un perimetro di rete logicamente isolato e dedicato in modo esclusivo al progetto. Per garantire alti standard di sicurezza (*Security by Design*), lo spazio di indirizzamento IP (CIDR) della VPC è stato rigorosamente segmentato in due livelli di *subnet*:

- *Subnet Pubbliche*: Esposte direttamente a Internet tramite un *Internet Gateway (IGW)*. Per privilegiare le logiche di automazione ed evitare l'overhead infrastrutturale di un *Bastion Host*, i nodi di calcolo EC2 (che formano il cluster Kubernetes self-managed basato su K3s, garantendo così assoluta parità architetturale con l'ambiente locale) e l'*Application Load Balancer (ALB)* sono stati istanziati in questa porzione di rete, ottenendo un IP pubblico per consentire il provisioning SSH diretto da parte di Ansible.

- *Subnet Private*: Completamente isolate dal traffico in ingresso proveniente da Internet. All'interno di queste subnet sicure sono stati relegati i database relazionali e NoSQL, proteggendo così i dati sensibili da qualsiasi tentativo di accesso esterno. Qualora queste risorse private necessitassero di comunicare verso l'esterno (es. aggiornamenti di sistema), il traffico in uscita verrebbe instradato in maniera unidirezionale attraverso un *NAT Gateway* posizionato appositamente nella subnet pubblica.

Questa topologia, unita a ferree regole di *Security Group* che limitano il traffico intra-subnet solo alle porte strettamente necessarie (es. porta 5432 per Postgres, 6379 per Redis), rende i componenti critici invulnerabili a tentativi di accesso diretto dall'esterno.

4.3 Offloading verso Servizi Gestiti AWS (PaaS/SaaS)

Nell'ecosistema cloud-native, gestire in proprio (su macchine virtuali EC2) infrastrutture come database o message broker è considerata una pratica sconsigliata (*anti-pattern*), a causa dell'ingente carico operativo richiesto per la manutenzione, il patching e i backup. Pertanto, i servizi precedentemente virtualizzati in locale sono stati migrati verso le controparti completamente gestite di AWS:

- *Amazon S3 (Storage Distribuito)*: Ha sostituito il file system locale per l'archiviazione delle immagini. S3 offre durabilità estrema (99.999999999%) e decupla il traffico statico dai web server.
- *Amazon RDS for PostgreSQL*: Ha sostituito il container Postgres locale. RDS fornisce backup automatici, crittografia a riposo (*at-rest*) e manutenzione automatica del motore di database, ospitando la tabella anagrafica dei task.
- *Amazon ElastiCache (Redis)*: Ha sostituito l'istanza Redis locale. Svolge la triplice funzione di cache ultraveloce per lo stato dei job (interrogata dal Frontend), di backend per Celery/memoria tem-

poranea, e soprattutto di *Distributed Lock* per serializzare l'accesso all'HPC universitario, come discusso nella Sezione 2.

- *Amazon SQS (Simple Queue Service)*: Ha sostituito RabbitMQ. Essendo un servizio *serverless*, SQS elimina la necessità di gestire cluster di code e dimensiona automaticamente il proprio *throughput* in base al traffico, fornendo code FIFO o Standard per lo smistamento dei job AI verso i Worker.

4.4 Risoluzione Criticità Hardware: Out-Of-Memory (OOM)

Durante i test di messa in produzione, è emersa una grave criticità hardware. Inizialmente l'infrastruttura era basata su **K3s**, una distribuzione Kubernetes estremamente leggera che lasciava sufficiente RAM libera per gestire i carichi dell'applicazione. Tuttavia, per simulare fedelmente un ambiente Enterprise standardizzato, si è scelto di fare l'upgrade al **Kubernetes nativo (kubeadm)**. Poiché i processi core di *kubeadm* richiedono autonomamente oltre 1.5 GB di memoria, lo spazio RAM residuo sulle limitate istanze **t3.small** (2 GB totali) del tier gratuito è divenuto pressoché inesistente.

Questa scelta architetturale ha generato un severo collo di bottiglia durante il caricamento in memoria della galleria dei grafi di scena: il file tensoriale grezzo (**test_gallery_scene_graphs.pt**) ha una dimensione di oltre 1.1 GB. L'importazione della libreria *PyTorch* e il caricamento diretto in RAM tramite le sue primitive richiedeva l'allocazione di ulteriore memoria, portando sistematicamente in *Out-Of-Memory* (OOM) le istanze EC2, causando crash irreversibili dei container (errori HTTP 502 Bad Gateway).

Un iniziale tentativo di *vertical scaling*, puntando a istanze **t3.medium** (4 GB di RAM), è stato precluso dalle restrizioni di budget e policy imposte dall'account *AWS Free Tier* (l'API cloud restituiva un **FreeTierRestrictionError**).

La soluzione architetturale definitiva ha perciò previsto la rimozione del caricamento dei tensori dai pod applicativi. Al suo posto, si è optato per una strategia di pre-calcolo (tramite un job locale) in cui l'intero dataset viene esportato in migliaia di leggeri file JSON, uno per ogni scena. Questi file sono stati sincronizzati direttamente su *Amazon S3*. Il backend è stato ri-programmato per interrogare il bucket S3 al volo (tramite protocollo HTTP/S3) recuperando la topologia del grafo richiesto on-demand. Questa modifica ha abbattuto drasticamente il *footprint* di memoria dei container, garantendo la totale stabilità dell'applicativo anche sulle macchine modeste fornite dal tier gratuito.

4.5 Innesco Serverless (Event-Driven) tramite AWS Lambda

Un ulteriore passo verso la decentralizzazione asincrona è stato raggiunto integrando *AWS Lambda*. Invece di affidare al Backend il compito di ricevere l'immagine e salvarla, il flusso è stato modificato in modo tale che il Frontend carichi l'immagine direttamente su un bucket S3.

Il caricamento del file (evento `s3:ObjectCreated`) innesca in modo invisibile, reattivo e *serverless* una funzione AWS Lambda. Quest'ultima estrae i metadati, inietta un nuovo messaggio nella coda SQS e termina la sua esecuzione in pochi millisecondi. Questa architettura ad eventi riduce ulteriormente le responsabilità del Backend FastAPI, garantendo un *decoupling* applicativo assoluto.

5 Deployment K3s Cloud ed Auto-Scaling Elastico

Una volta consolidata l'infrastruttura sottostante su AWS, l'ultimo tassello dell'architettura ha riguardato il *deployment* effettivo dei microservizi all'interno del cluster Kubernetes self-managed (K3s) e la configurazione delle logiche di scalabilità automatica, essenziali per far fronte a

carichi di lavoro variabili tipici delle applicazioni cloud-native.

5.1 Gestione dei Secret e Manifesti Kubernetes

Il deployment applicativo è stato dichiarato tramite manifesti YAML standard di Kubernetes (**Deployment**, **Service**, **Ingress**). Particolare attenzione è stata rivolta alla sicurezza e all'isolamento delle credenziali. Parametri sensibili quali le stringhe di connessione a RDS e Redis, i token di accesso alle API AWS e, in particolar modo, le chiavi private SSH necessarie al Worker per autenticarsi sul cluster HPC, non sono mai stati inseriti in chiaro nel codice sorgente. Sono stati invece iniettati dinamicamente all'interno di oggetti **Secret** di Kubernetes, montati a runtime come variabili d'ambiente o come volumi cifrati (*tmpfs*) all'interno dei container.

Il traffico HTTP in ingresso è stato esposto su Internet tramite un *Ingress Controller*, il quale è stato automaticamente associato da AWS a un Application Load Balancer (ALB) pubblico, che si occupa di smistare il traffico sui pod del Frontend e del Backend allocati sui nodi K3s. Sebbene i nodi EC2 risiedano nella subnet pubblica (per ragioni di automazione SSH), il loro *Security Group* è configurato per negare qualsiasi connessione HTTP (porta 80) proveniente direttamente da Internet. Il traffico sulla porta 80 è consentito esclusivamente se originato dall'interno della VPC. Di conseguenza, l'utente finale non può by-passare il bilanciatore: è fisicamente obbligato a inviare la richiesta all'ALB, il quale fungerà da scudo e inoltrerà la connessione in modo sicuro ai nodi sottostanti sfruttando il routing interno della VPC.

5.2 Event-Driven Auto-Scaling con KEDA

La gestione delle risorse computazionali per i workload di intelligenza artificiale pone una sfida unica: l'elaborazione di un'immagine può consumare tempi e risorse imponenti. L'approccio classico di Kubernetes all'autoscaling, l'*Horizontal Pod Autoscaler* (HPA), si basa principal-

mente su metriche hardware (percentuale di CPU o RAM utilizzata). Tuttavia, nel nostro scenario asincrono, un picco di traffico si traduce in un improvviso accumulo di messaggi nella coda SQS, non in un aumento istantaneo della CPU del Worker (il quale può processare un solo messaggio alla volta).

Per risolvere questo disallineamento, si è abbandonato l'HPA standard in favore di *KEDA* (*Kubernetes Event-Driven Autoscaling*). KEDA è in grado di agire da ponte tra Kubernetes e i provider cloud, interrogando direttamente le metriche dei Message Broker esterni.

Nello specifico, è stato configurato un `ScaledObject` che istruisce Kubernetes a monitorare la coda Amazon SQS. Il parametro cruciale per il tuning di questa operazione è stato il `queueLength`. Impostando questo valore a 1, si è comunicato al cluster una regola aggressiva ed elastica: per ogni singolo messaggio presente in coda, KEDA deve effettuare lo **scale-out** (generare) di un nuovo pod Worker, fino a un massimo preimpostato (*maxReplicaCount*).

In assenza di messaggi (coda vuota), KEDA scala il Deployment a *zero repliche* (*scale-to-zero*), abbattendo completamente i costi infrastrutturali nei periodi di inattività, realizzando così il vero potenziale del paradigma Serverless applicato ai container.

L'interazione tra KEDA e il *Distributed Lock* su Redis (discusso nel Capitolo 2) rappresenta un punto chiave di questo sistema: l'architettura è libera di scalare orizzontalmente in modo massiccio per aggredire una coda SQS piena di immagini da processare in parallelo, delegando simultaneamente la sincronizzazione fine e la serializzazione delle risorse GPU al semaforo logico gestito in-memory.

A titolo dimostrativo, si riporta in Figura 5 un log reale del cluster registrato durante un test di stress, in cui l'HPA governato da KEDA effettua uno scale-out reattivo portando le repliche dei worker applicativi da 1 a 3 a causa di un improvviso accumulo di task sulla coda SQS.

```
$ kubectl get hpa -n default
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
keda-hpa-worker-sqs-scaler	Deployment/worker	667m/1 (avg)	1	10	3


```
$ kubectl get pods -n default
```

NAME	READY	STATUS	AGE	
worker-7d85889bf5-7w4cn	1/1	Running	16m	
worker-7d85889bf5-47vsr	1/1	Running	77s	<-- (Nuovo pod scalato da KEDA)
worker-7d85889bf5-njzfh	1/1	Running	62s	<-- (Nuovo pod scalato da KEDA)

Figura 5: Esempio di Autoscaling dinamico Event-Driven registrato live dal cluster Kubernetes.

5.3 Continuous Integration e Continuous Deployment (CI/CD)

A coronamento del ciclo di vita del software, è stata implementata una pipeline di automazione basata su *GitHub Actions*. Qualsiasi modifica al codice sorgente spinta (tramite *git push*) sul branch principale innesca automaticamente un flusso di lavoro diviso in due macro-fasi:

- *Build & Publish*: L'infrastruttura GitHub compila le nuove immagini Docker per i microservizi aggiornati, iniettando le dipendenze fresche, e le deposita in un Container Registry sicuro (GHCR).
- *Rolling Update*: La pipeline si connette in modo autenticato al cluster Kubernetes su AWS, aggiornando l'hash dell'immagine nei Deployment. Kubernetes avvia quindi un *Rolling Update*, spegnendo i vecchi pod e accendendo quelli nuovi in maniera graduale, garantendo uno *Zero-Downtime Deployment* e assicurando che l'utente finale non percepisca alcuna interruzione del servizio durante gli aggiornamenti.

Questo approccio GitOps chiude il cerchio dell'infrastruttura Cloud-Native, fornendo un sistema robusto, osservabile, elastico e completamente automatizzato, in grado di portare modelli AI accademici e pesanti in un ambiente di produzione scalabile.

5.4 Riproducibilità e Automazione del Provisioning

Al fine di garantire la totale riproducibilità dell'infrastruttura (sia in ambiente simulato tramite K3s che in cloud), l'intero processo di provisioning è stato incapsulato in script di automazione (`deploy_cloud.sh` e `setup_multipass.sh`) che si interfacciano in maniera trasparente con Terraform, Ansible e le interfacce a riga di comando (CLI).

Per effettuare un'installazione dell'architettura *from scratch*, il deployer è tenuto a soddisfare una rigorosa serie di prerequisiti:

- *Toolchain di sistema*: Presenza di Terraform, Ansible, AWS CLI e GitHub CLI (`gh`) sull'host operativo.
- *Configurazione dell'Ambiente*: Compilazione del manifest `project_config.env` contenente i mount-path locali per i dati e le credenziali di base (es. password RDS).
- *Autenticazione CLI*: Autenticazione attiva verso l'account AWS (le cui sessioni devono essere valide, requisito cruciale in ambienti protetti come gli AWS Learner Lab che invalidano le token frequentemente) e verso GitHub tramite `gh auth`. Quest'ultimo è essenziale affinché lo script possa iniettare autonomamente l'IP delle macchine AWS e le chiavi SSH nei *Secret* della repository senza intervento umano.
- *Inizializzazione Infrastructure-as-Code*: Download dei provider cloud e crittografici necessari per Terraform (`terraform init`).
- *Sincronizzazione Dati Pesanti*: Lo script master gestisce in blocco anche la remotizzazione dei modelli PyTorch, dell'indice FAISS e delle immagini di test verso i bucket S3 (`aws s3 sync`).

Una volta verificato l'ambiente, è sufficiente lanciare lo script principale dalla root del progetto:

```
$ ./deploy_cloud.sh
```

L'esecuzione del file batch provvede autonomamente all'orchestrazione end-to-end: inizializza ed applica Terraform per la creazione della VPC e delle macchine su AWS, estrae gli IP per aggiornare l'inventario Ansible, remotizza i modelli pesanti su S3 ed infine agisce da *trigger* per le GitHub Actions. Questa catena garantisce che il passaggio dallo strato *Local* all'ambiente di *Production* nel cloud avvenga interamente in maniera dichiarativa e automatizzata.

Terminati i test sul cloud, per evitare l'addebito di costi imprevisti, è sufficiente abbattere tutte le risorse allocate con l'istruzione di *destroy*:

```
$ cd cloud/deploy/aws/terraform  
$ terraform destroy --auto-approve
```

6 Conclusioni e Sviluppi Futuri

L'ingegnerizzazione di un sistema di *Scene Understanding* e retrieval semantico distribuito ha evidenziato in maniera chiara il potenziale, ma anche l'incredibile complessità, dello sviluppo Cloud-Native moderno. Il progetto, nato inizialmente come script locale fortemente limitato e sincrono, è stato frammentato ed elevato a un'infrastruttura Enterprise pronta alla produzione.

Il raggiungimento dei tre milestone ha portato a risultati concreti:

- *Decoupling Perfetto*: L'adozione del pattern asincrono tramite broker di messaggistica ha salvato il frontend web dal collasso sotto carichi pesanti, fornendo al contempo feedback intermedi all'utente finale.
- *Gestione del Codice Universale*: Sfruttando Terraform e Ansible, si è dimostrato come l'intero strato infrastrutturale possa essere documentato, versionato su Git e riprodotto ciecamente in una manciata di minuti, annullando il rischio di derivazioni configurazionali o errori manuali (Configuration Drift).

- *Ottimizzazione Finanziaria (FinOps)*: Scontrandosi attivamente con i limiti fisici imposti dal Free Tier di AWS (istanze `t3.micro`), il progetto è riuscito a eseguire modelli complessi adattando l'architettura all'uso di *K3s*. La rinuncia a un cluster Kubernetes standard ha confermato come la flessibilità architetturale sia vitale per contenere i costi computazionali.

6.1 Sviluppi Futuri

Nonostante il sistema sia stabile e testato, l'architettura attuale pone le basi per futuri sviluppi e raffinamenti, in particolare in ottica di estrema scalabilità:

1. *Rafforzamento della Sicurezza (Subnet Private per i Nodi)*: Attualmente i nodi EC2 del cluster Kubernetes sono situati nelle subnet pubbliche per facilitare l'accesso SSH diretto e il provisioning tramite Ansible. Uno sviluppo futuro cruciale prevede lo spostamento dei nodi computazionali all'interno delle subnet private, limitando l'accesso dall'esterno tramite l'utilizzo di un *Bastion Host* o del servizio *AWS Systems Manager (SSM)*, incrementando nettamente il livello di sicurezza.
2. *Autoscaling dell'Infrastruttura Fisica (Cluster Autoscaler)*: Sebbene il sistema implementi già KEDA per scalare dinamicamente il numero di Pod in base ai messaggi in coda, le macchine fisiche (EC2) sottostanti rimangono statiche. Integrando strumenti come *Karpenter* o *Kubernetes Cluster Autoscaler*, si permetterebbe ad AWS di accendere e spegnere automaticamente nuove istanze EC2 qualora i nodi attuali si saturassero, ottimizzando ulteriormente il bilanciamento tra costi e performance.
3. *Indipendenza dal Cluster HPC Universitario*: L'architettura attuale dipende dal cluster HPC esterno per l'esecuzione del Vision-Language Model (VLM). Questo introduce un collo di bottiglia dovuto alla disponibilità del nodo e alla necessità del *Distributed*

Lock. Una futura migrazione verso nodi EC2 dotati di acceleratori hardware (famiglia **g4dn** con GPU NVIDIA) permetterebbe di inglobare il VLM direttamente nel cloud, eliminando la dipendenza esterna e parallelizzando l'inferenza.

4. *Ottimizzazione Cache tramite Hashing:* Attualmente, il sistema di caching Redis verifica lo stato tramite i Job ID. Implementando funzioni di hash crittografico sul contenuto binario delle immagini, si potrebbe innescare un *Cache HIT* per immagini identiche caricate più volte, risparmiando ulteriore computazione.

In definitiva, questo elaborato dimostra come la transizione da codice di ricerca accademico (modelli Deep Learning sperimentali) a prodotto software as-a-Service solido e commerciale passi inevitabilmente attraverso rigorose best practice di Ingegneria del Software, Orchestrazione Container e Cloud Computing.